ACI ASSIGNMENT - AIMLCZG557/AECLZG557

- **Assignment Title:** Smart Waste Collection Agent - IDA* Search **Group ID:** G225

The PEAS Analysis

1. Performance Measure

This Smart Waste Collection robot has one main job: transport the waste from the starting collection center to the processing plant.

The criteria used to evaluate how successful the agent is at achieving its goal.

- **Primary Objective:** Successfully reach the target waste processing unit (destination) from the starting collection center (source).
- **Cost Minimization:** Minimize the overall travel cost (which represents distance, fuel consumption, or battery usage).
- **Navigation Efficiency:** Avoid unnecessary travel and minimize the number of nodes explored during the search.
- **Resource Management:** Operate within the limited battery power and fuel efficiency constraints.
  Avoid wandering around or overthinking the route. It should minimize the number of intersections (nodes).

2. Environment

The robot is navigating the streets of Bengaluru, The waste centers and processing plants are the stops (vertices - e.g., MG_Road, Yelahanka), the roads connecting them are the paths (edges), and the travel cost between any two stops is the weight.

The world or conditions in which the agent operates:

- **Environment Properties:**
  - *Fully Observable:* The agent has access to the complete map (the entire graph and its weights) prior to making decisions.
  - *Deterministic:* Moving along a specific edge will consistently result in the same cost and lead to the same destination node.
  - *Static:* The map and road networks do not change while the agent is calculating the route.
  - *Discrete:* There is a finite number of locations (nodes) and roads (edges) to choose from.

3. Actuators

These are the physical components that allow the robot to act on the routing decisions it makes.

- **Movement:** The physical drivetrain, motors to move the robot along the roads between the collection centers.
- **Steering:** The internal controllers that physically turn the robot to keep it on the optimal path generated by the algorithm.
- **Waste Handling:** Knowing that the problem focuses heavily on navigation, a waste collection robot would also utilize robotic arms or dump mechanisms to pick up and drop off the garbage.

4. Sensors

The devices the agent uses to perceive its environment and gather data.

To execute the plan, the robot needs constant feedback from its environment and its own systems.

- **Navigation:** A GPS module or location tracker so it knows exactly where it is on the map and recognizes when it has reached the processing unit.

- **Vitals:** Battery and fuel monitors to ensure it has enough power to survive the calculated trip.
- **Inputs:** A data receiver to accept its daily movement orders from the user—specifically, knowing its starting point and final destination.

Search Strategy: Heuristic, Algorithm and Cost

Heuristic

To optimize the pathfinding efficiency of the Smart Waste Collection Robot Agent, the Iterative Deepening A* algorithm utilizes a custom heuristic to estimate the remaining travel cost from any current node to the destination processing plant.

Heuristic Chosen

Let:

- $h(n)$= estimated cost from current node nnn to the goal node.
- d_{min} = minimum edge cost in the graph.
- $hops(n,\ goal)$= minimum number of edges between node nnn and the goal (ignoring weights).

Then: $$h(n) = hops(n, goal) \times d_{min}$$

The heuristic corresponds to solving a relaxed version of the problem, which is - ignore the actual edge weights and assume every edge costs the minimum edge cost in the graph.

The heuristic must be compatible with **IDA*** and should be **admissible**.The reason for choosing this is, we observe that every road has a positive travel cost and cheapest possible road in the entire graph has cost d_{min}. To reach the destination, the robot must traverse at least $hops(n, goal)$ roads.

Admissibility check

The robot must traverse at least $hops(n, goal)$ number of edges to reach the destination, and because no edge in the entire network can cost less than d_{min}, the product of these two values represents the minimum cost to complete the journey.

Consistency

The heuristic is consistent, meaning $h(n) \leq c(n, m) + h(m)$. This holds because the actual step cost, $c(n, m)$, is always at least d_{min}, which perfectly offsets the maximum one-hop drop in our remaining estimated cost.

Algorithm Choice (IDA*)

Iterative Deepening A* (IDA*) was chosen as the informed search algorithm for this problem because it combines the optimal pathfinding capability of A* with the low memory requirements of Depth-First Search. IDA* uses the evaluation function $f(n) = g(n) + h(n)$ to guide the search toward the goal while iteratively increasing a cost threshold. Unlike A*, which stores all generated nodes in memory, IDA* performs a depth-first traversal and therefore requires significantly less memory. This makes it particularly suitable for a Smart Waste Collection Robot operating with limited computational resources and battery power. When used with an admissible heuristic, IDA* guarantees finding the optimal route while efficiently minimizing travel cost.

Cost Function

The IDA* algorithm evaluates each node using the cost function $f(n) = g(n) + h(n)$. Here, (g(n)) represents the actual travel cost accumulated from the source node to the current node, calculated as the sum of the edge weights along the path traversed so far. The heuristic component (h(n)) estimates the remaining cost from the current node to the destination. By combining the cost already incurred with the estimated future cost, the algorithm efficiently guides the search toward the least-cost path while avoiding unnecessary exploration.

Alternative Problem Modeling & Performance Implications

An alternative way to model the Smart Waste Collection problem is to use the standard A* search algorithm instead of IDA*. The waste management network is still represented as a weighted graph, and routes are evaluated using the same evaluation function: $f(n) = g(n) + h(n)$

where g(n) is the actual travel cost from the source to node(n), and h(n) is the heuristic estimate from (n) to the destination. Unlike IDA*, A* maintains an Open List (priority queue/min-heap) and a Closed List, always expanding the node with the lowest estimated total cost.

Performance Implications

Time Efficiency - A* generally finds the optimal path faster because it stores the search frontier and avoids the repeated depth-limited searches performed by IDA*. As a result, fewer nodes are re-expanded, reducing execution time.

Memory Consumption - The major drawback of A* is its high memory requirement. It stores all generated and explored nodes, resulting in a space complexity of $O(b^d)$, where (b) is the branching factor and (d) is the depth of the optimal solution. For a large city road network, the memory usage can grow rapidly.

Justification for IDA* - IDA* performs iterative deepening using depth-first traversal and requires only $O(bd)$ memory. Although some nodes may be re-expanded across iterations, its significantly lower memory usage makes it more suitable for a battery-powered Smart Waste Collection Robot with limited onboard computational resources. Therefore, IDA* provides a better balance between optimality and memory efficiency for large-scale route planning.

Implementation Details

Data Structures Used

The waste collection network is represented as a weighted graph using an Adjacency List implemented with Python's `defaultdict(list)`. Each node stores a list of tuples containing its neighboring node and the corresponding travel cost. A separate set is maintained to store all valid node names, enabling efficient node validation and membership checking.

The (IDA*) algorithm performs a bounded Depth-First Search (DFS) using Python's recursion mechanism. The current route from source to destination is maintained as a path list during traversal, while the recursion stack naturally supports backtracking. This design allows (IDA*) to achieve low memory consumption because only the current search path is stored at any time.

Error Handling

Basic error handling has been incorporated to ensure reliable execution. During input parsing, the program validates the presence of both Source and Destination fields and skips incomplete test cases with appropriate messages. It also verifies that the specified source and destination nodes exist in the graph before initiating the search. Invalid edge definitions are detected during graph construction and reported to the user. If the destination is unreachable due to a disconnected graph, the algorithm returns no valid path and reports the failure gracefully instead of terminating unexpectedly. Unreachable states are represented using `float('inf')`.

Results and Output

The implemented IDA* algorithm was tested using the sample waste collection networks. For each test case, the program successfully identified the least-cost route between the specified source and destination locations.

The output generated by the program includes:
Optimal Path Found: Displays the sequence of locations forming the least-cost route.
Total Travel Cost: Reports the cumulative edge cost that is with the optimal path.
Number of Nodes Explored: Indicates the total number of nodes expanded, providing a measure of search effort.
Sequence of Locations Visited: Displays the order in which locations were explored.

The following output shows sample executions of the program.

```
Reading input from 'inputPS1.txt' …
--- Case 1: MG_Road -> Yelahanka ---
--- Case 2: A -> E ---

SOURCE: MG_Road
DESTINATION: Yelahanka
OPTIMAL PATH FOUND: MG_Road → Electronic_City → Whitefield → Yelahanka
ROUTE DETAILS:
├─ Total Travel Cost: 8 units
├─ Nodes Explored: 10 nodes
└─ Path Length: 3 hops
SEQUENCE OF VISITED LOCATIONS:
1. MG_Road
2. Electronic_City
3. Whitefield
4. Yelahanka

SOURCE: A
DESTINATION: E
OPTIMAL PATH FOUND:
A → C → D → E
ROUTE DETAILS:
├─ Total Travel Cost: 5 units
├─ Nodes Explored: 22 nodes
└─ Path Length: 3 hops
SEQUENCE OF VISITED LOCATIONS:
1. A
2. C
3. D
4. E
[Output written to 'outputPS1.txt']
```